



Optimización de Consultas SQL

SUSANA MEDINA G.

SUSANA.MEDINA@CORREOUNIVALLE.EDU.CO

Ejemplo: Base de datos de una empresa

```
CREATE TABLE PROJECT
(
    Pname          VARCHAR(20) ,
    Pnumber        NUMERIC(3) ,
    Plocation      VARCHAR(20) ,

    PRIMARY KEY (Pnumber)
);
```

Index

Los **índices** son tablas de **búsqueda** que permiten al motor de búsqueda de la base de datos acelerar la obtención de datos.

En otras palabras un índice es un puntero en una tabla.

Index

Index

Los **índices** son tablas de **búsqueda** que permiten al motor de búsqueda de la base de datos acelerar la obtención de datos.

En otras palabras un índice es un puntero en una tabla.

Ventaja: Acelera las consultas SELECT y sentencias WHERE.

Desventaja: Hace lenta la inserción de datos en las sentencias UPDATE e INSERT.

Índex: sintaxis

```
CREATE INDEX index_name ON table_name;
```

```
CREATE INDEX name ON table USING HASH (column);
```

Tipos de índices

PostgreSQL tiene los siguientes tipos de índices:

- ★ B-tree (por defecto)
- ★ Hash
- ★ GiST
- ★ SP-GiST
- ★ GIN

Evitar índices cuando ...

- ❖ Las tablas son **pequeñas**
- ❖ Las tablas tienen frecuentemente *large batch update* u operaciones **INSERT**.
- ❖ Hay una columna que contenga un gran número de valores **NULL**.

Parallel Queries

Consultas paralelas

En PostgreSQL es posible distribuir las *queries* para hacer uso de **múltiples CPUs** (núcleos) para responderlas más **rápido**.

Para las queries que pueden beneficiarse:

- El **doble** de velocidad (normal)
- **Cuatro** veces más veloces o más (algunas).

Consejos rendimiento

Qué hacer si
hay una
consulta que se
demora
mucho?

Sentencia
EXPLAIN

Sentencia EXPLAIN

Este comando permite ver el plan de la *query* creado por el **planner** para cualquier *query*.

El planner es la parte del SGBD encargada de crear los planes para las *queries* (consultas).

La estructura del plan de la *query* es un **árbol** de nodos tipo plan.

Sentencia EXPLAIN: ejemplo 1

```
EXPLAIN SELECT * FROM tenk1;
```

QUERY PLAN

```
Seq Scan on tenk1 (cost=0.00..458.00 rows=10000 width=244)
```

Sentencia EXPLAIN

Costo inicial estimado (*Estimated start-up cost*). Tiempo antes de la fase de output (ordenamiento).

Costo total estimado (*Estimated total cost*). Tiempo que le toma ejecutarse en su totalidad al plan de la query (cuando para el nodo padre -> todas las filas).

Número estimado de filas output: se asume que este nodo se ejecuta completamente.

Ancho promedio estimado del output de filas (en *bytes*).

Sentencia EXPLAIN: ejemplo 2

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 7000;
```

QUERY PLAN

```
Seq Scan on tenk1  (cost=0.00..483.00 rows=7001 width=244)  
  Filter: (unique1 < 7000)
```

Sentencia EXPLAIN: ejemplo 3

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100;
```

QUERY PLAN

```
Bitmap Heap Scan on tenk1 (cost=5.07..229.20 rows=101 width=244)  
  Recheck Cond: (unique1 < 100)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04  
rows=101 width=0)  
      Index Cond: (unique1 < 100)
```

Sentencia EXPLAIN: ejemplo 4

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND stringu1 =  
'xxx';
```

QUERY PLAN

```
Bitmap Heap Scan on tenk1 (cost=5.04..229.43 rows=1 width=244)  
  Recheck Cond: (unique1 < 100)  
  Filter: (stringu1 = 'xxx'::name)  
    -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04  
rows=101 width=0)  
      Index Cond: (unique1 < 100)
```

Sentencia EXPLAIN: ejemplo 5

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 = 42;
```

QUERY PLAN

```
Index Scan using tenk1_unique1 on tenk1 (cost=0.29..8.30  
rows=1 width=244)
```

```
Index Cond: (unique1 = 42)
```

Sentencia EXPLAIN: ejemplo 6

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 ;
```

QUERY PLAN

```
Bitmap Heap Scan on tenk1 (cost=25.08..60.21 rows=10 width=244)  
  Recheck Cond: ((unique1 < 100) AND (unique2 > 9000))  
    -> BitmapAnd (cost=25.08..25.08 rows=10 width=0)  
      -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0)  
          Index Cond: (unique1 < 100)  
      -> Bitmap Index Scan on tenk1_unique2 (cost=0.00..19.78 rows=999 width=0)  
          Index Cond: (unique2 > 9000)
```

Sentencia EXPLAIN: ejemplo 7

```
EXPLAIN SELECT * FROM tenk1 WHERE unique1 < 100 AND unique2 > 9000 LIMIT 2;
```

QUERY PLAN

```
Limit (cost=0.29..14.48 rows=2 width=244)
```

```
  -> Index Scan using tenk1_unique2 on tenk1 (cost=0.29..71.27 rows=10 width=244)
```

```
    Index Cond: (unique2 > 9000)
```

```
    Filter: (unique1 < 100)
```

Sentencia EXPLAIN: ejemplo 8 (JOIN)

```
EXPLAIN SELECT * FROM tenk1 t1, tenk2 t2
WHERE t1.unique1 < 10 AND t1.unique2 = t2.unique2;
```

QUERY PLAN

```
-----
Nested Loop(cost=4.65..118.62 rows=10 width=488)
-> Bitmap Heap Scan on tenk1 t1  (cost=4.36..39.47 rows=10 width=244)
    Recheck Cond: (unique1 < 10)
    -> Bitmap Index Scan on tenk1_unique1  (cost=0.00..4.36 rows=10
width=0)
        Index Cond: (unique1 < 10)
    -> Index Scan using tenk2_unique2 on tenk2 t2  (cost=0.29..7.91 rows=1
width=244)
        Index Cond: (unique2 = t1.unique2)
```

Índice B-tree (árbol autobalanceado)

El *query planner* usará estos índices cuando la consulta involucre algunos de estos **operadores**:

<

<=

=

>=

BETWEEN

IN

IS NULL

IS NOT NULL

Índice B-tree

El *query planner* también usará estos índices para resolver consultas con los operadores **LIKE** y **~**, como:

- `column_name LIKE 'foo%'`
- `column_name LIKE 'bar%'`
- `column_name ~ '^foo'`

Índice GIN

GIN significa: **g**eneralized inverted **in**dexes.

Son útiles para **valores múltiples** almacenados en una sola columna. Por ejemplo:

- hstore
- array
- jsonb
- range types

Índice BRIN

BRIN significa **b**lock **r**ange **i**ndexes.

Es más pequeño y menos costoso en comparación que el B-tree index.

BRIN permite el uso de un índice en una tabla muy grande que sería impráctica usando el B-tree sin partición horizontal.

Usualmente se usa en una columna que ha sido ordenada de forma lineal. Por ejemplo: la columna *created date* de la tabla *sales order*.

Índice GiST

GiST significa **G**eneralized **S**earch **T**ree.

Los índices **GiST** permiten construir estructuras de tipo árbol generales.

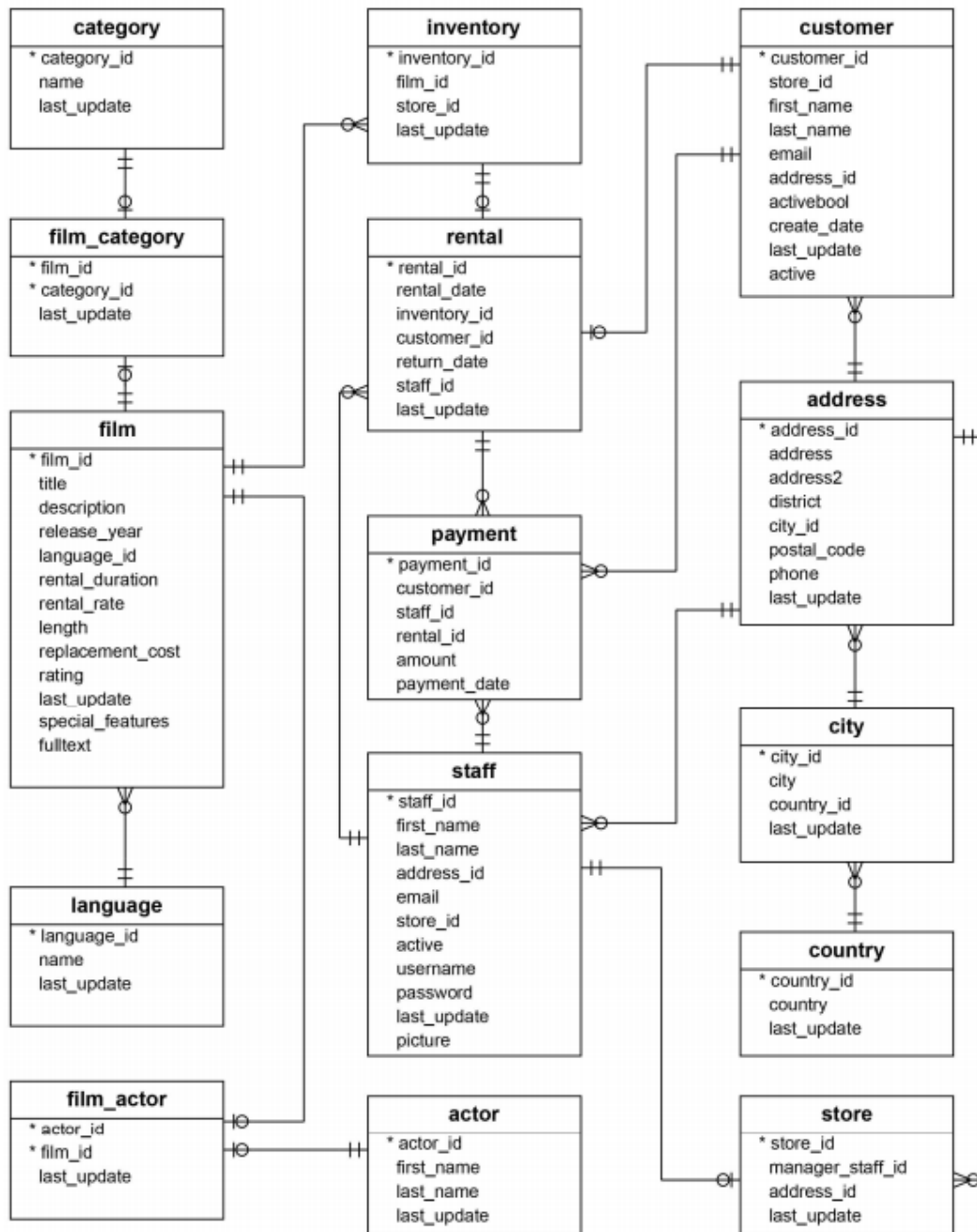
Son útiles para indexar datos de tipo geométricos y búsquedas de texto.



Sentencia

EXPLAIN

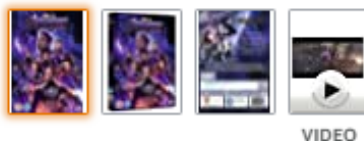
ANALYZE



MER DVDRENTAL



haz clic en la imagen para obtener una vista ampliada



Avengers Endgame [DVD] [2019]

Formato: DVD

★★★★★ ∨ 10,078 calificaciones

Precio: **US\$ 17.99** + Sin depósito de derechos de importación y US\$ 11.98 de envío a Colombia (**Envío GRATIS** a Colombia cuando gastes más de US\$ 35.00 en artículos aptos) [Detalles](#)

Blu-ray
US\$ 22.98

DVD
US\$ 17.99

3D
US\$ 37.95

4 K
—

Opciones de DVD adicionales	Edición	Discos	Precio de Amazon	Nuevo desde	Usado desde
DVD	—	1	US\$ 17.99	US\$ 10.47	US\$ 7.99

Disponible a un precio menor de [otros vendedores](#) que podrían no ofrecer envío Prime gratis.

[Reportar información de producto incorrecta](#)

! Región de Reproducción 2 :Este DVD no se podrá reproducir en algunos de los reproductores de DVD vendidos en México. El artículo requiere un reproductor de DVD específico de la región o multirregión y una TV compatible [aquí](#)

```
dvdrental=# explain analyze select first_name, title from actor, film, film_actor
dvdrental=# where last_name='Cage' and
dvdrental=# actor.actor_id = film_actor.actor_id and
dvdrental=# film.film_id = film_actor.film_id
dvdrental=# ;
```

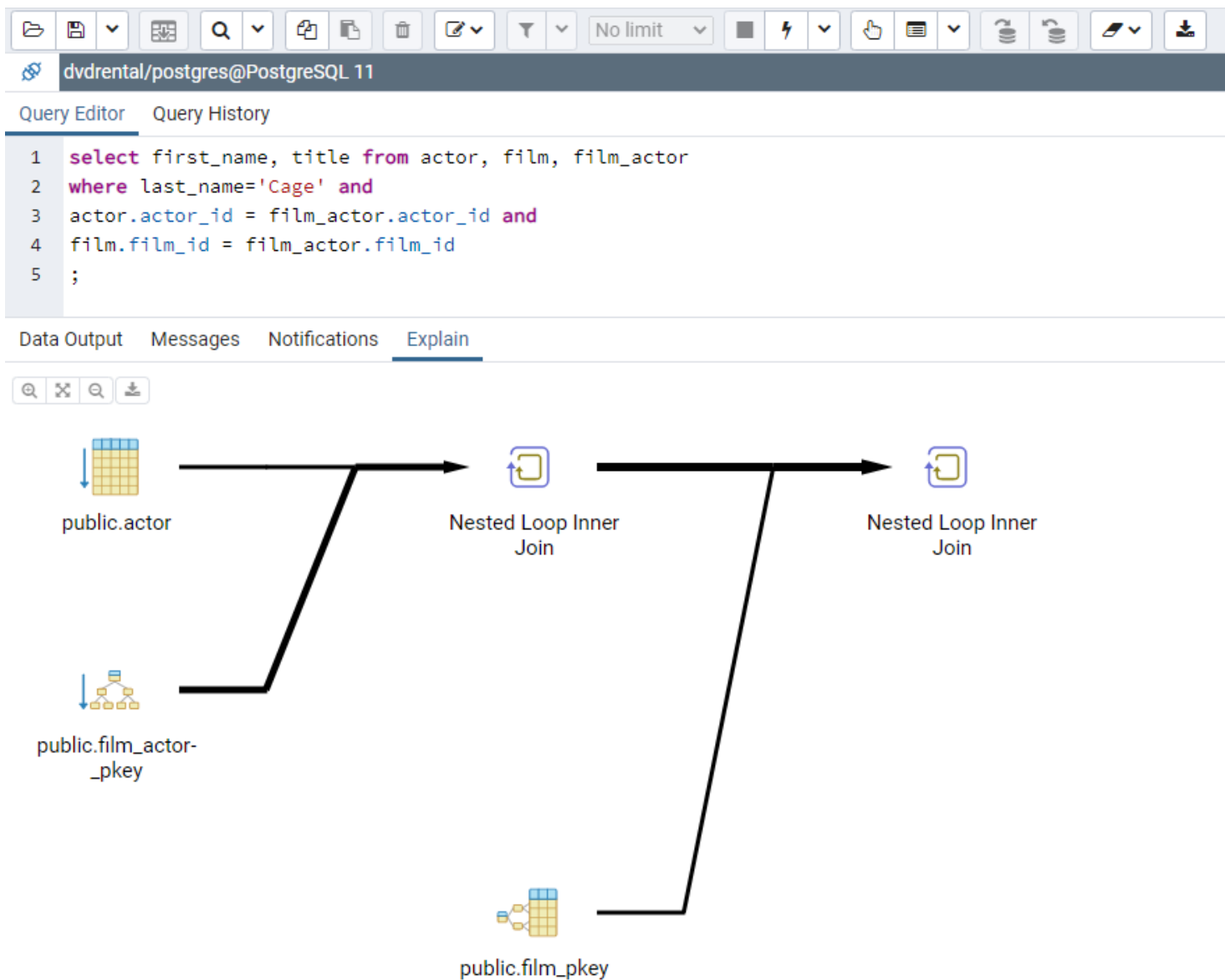
QUERY PLAN

```
-----
Nested Loop (cost=0.56..86.51 rows=49 width=21) (actual time=0.030..0.167 rows=54 loops=1)
-> Nested Loop (cost=0.28..70.06 rows=49 width=8) (actual time=0.026..0.064 rows=54 loops=1)
    -> Seq Scan on actor (cost=0.00..4.76 rows=2 width=10) (actual time=0.018..0.038 rows=2 loops=1)
        Filter: ((last_name)::text = 'Cage'::text)
        Rows Removed by Filter: 219
    -> Index Only Scan using film_actor_pkey on film_actor (cost=0.28..32.38 rows=27 width=4) (actual time=0.005..0.009 rows=27 loops=2)
        Index Cond: (actor_id = actor.actor_id)
        Heap Fetches: 54
-> Index Scan using film_pkey on film (cost=0.28..0.34 rows=1 width=19) (actual time=0.002..0.002 rows=1 loops=54)
    Index Cond: (film_id = film_actor.film_id)
Planning Time: 0.625 ms
Execution Time: 0.195 ms
(12 filas)
```

Power Shell Windows

EXPLAIN ANALYZE

DVDRENTAL



pgAdmin

EXPLAIN ANALYZE

DVDRENTAL

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Nested Loop Inner Join (cost=0.56..88.26 rows=55 width=2...	0.042 ms	0.689 ms	↑ 1.02	54	55	1
2.	→ Nested Loop Inner Join (cost=0.28..69.8 rows=55 widt...	0.026 ms	0.269 ms	↑ 1.02	54	55	1
3.	→ Seq Scan on public.actor as actor (cost=0..4.5 row... Filter: ((actor.last_name)::text = 'Cage'::text) Rows Removed by Filter: 198	0.156 ms	0.156 ms	↑ 1	2	2	1
4.	→ Index Only Scan using film_actor_pkey on public.fil... Index Cond: (film_actor.actor_id = actor.actor_id)	0.088 ms	0.088 ms	↑ 1	27	27	2
5.	→ Index Scan using film_pkey on public.film as film (cost... Index Cond: (film.film_id = film_actor.film_id)	0.378 ms	0.378 ms	↑ 1	1	1	54

Tabla con el query plan - pgAdmin

EXPLAIN ANALYZE

DVDRENTAL

Ejercicio Práctico



Recordando las cláusulas...

```
SELECT <atributos y funciones de agregación>  
FROM <lista de tablas>  
[ WHERE <condiciones> ]  
[ GROUP BY <atributos de agrupamiento> ]  
[ HAVING <condición a nivel de grupo> ]  
[ ORDER BY <atributos> ];
```

Referencias

Página con documentación de PostgreSQL 9.1:

<https://www.postgresql.org/docs/9.1>

fecha consulta: Octubre 30, 2019